

Which SQL injection vulnerabilities should you fix first?

Given that not all vulnerabilities can be fixed, how can a company detect, prioritize, and block SQL injection attacks?

 **Imam MAGADIYEV**
imam.magadiyev@etu.iut-tlse3.fr

 **Yeadonaye SENTAYEHU**
yeadonaye-ashenafi.sentayehu@etu.iut-tlse3.fr

Abstract

SQL injection remains one of the most persistent database security threats, with over 62,000 recorded CVEs, far more than any team can realistically remediate. We present a hybrid approach combining machine-learning-based SQL injection detection with CVE-driven vulnerability prioritization, enabling organizations to detect, rank, and block attacks efficiently. Using a dataset of 457,233 network traffic samples (70,023 SQLi and benign payloads), our detection model achieves a 97% average F1-score. The prioritization strategy further directs remediation toward the most exploitable vulnerabilities, helping teams focus limited resources where they matter most.

Introduction

Database security remains a major challenge. The Injection category accounts for 62,445 CVEs, including more than 14,000 specifically related to SQL injection. With such a large number of vulnerabilities, a company cannot realistically fix them all at once. The key question is therefore: which SQL injection vulnerabilities should be fixed first? Existing approaches often focus on detecting attacks without integrating prioritization and prevention. This project combines machine-learning-based detection, CVE-based prioritization, and attack blocking to focus remediation efforts on the most exploitable vulnerabilities.

Methods

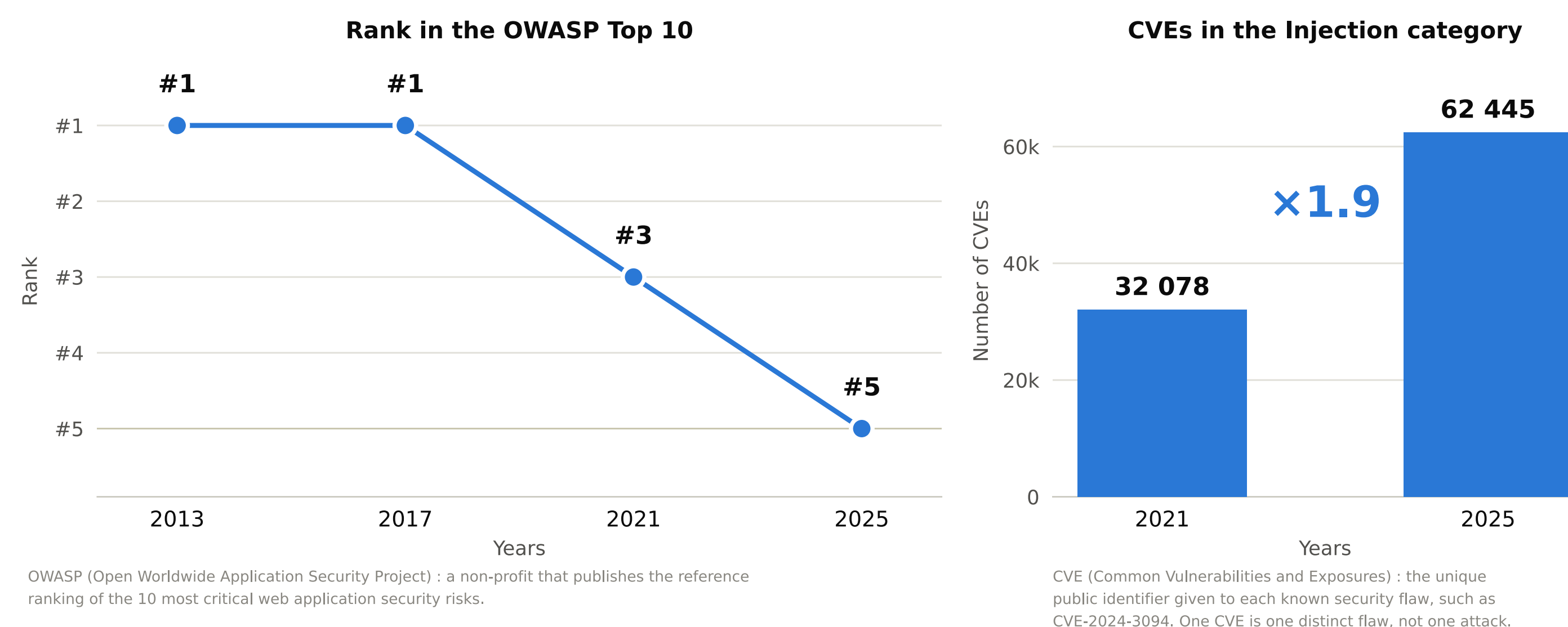
Synthesis of a peer-reviewed CNN-LSTM detection framework (Paul et al., 2024) trained on 457,233 network traffic samples and 70,023 SQLi/benign payloads, using TF-IDF and GloVe embeddings. Payload data split 60:40, NetFlow data split 70:30. Prioritization tested on 317 SQLi CVEs (NVD, 2020-2023), scoring severity (CVSS), exploitation likelihood (EPSS), and Expected Exploitability (EE); coverage and additional effort compared across metrics.

Discussion

Detection performed well across models, with CNN-LSTM reaching 97% F1-score on both payload and network traffic data. Prioritization revealed a coverage-effort tradeoff: CVSS alone flagged 259 of 317 CVEs but with high wasted effort, while EPSS alone flagged only 19, missing most exposure. Combining CVSS, EPSS and EE narrowed this to 153 CVEs with minimal wasted effort, suggesting single-metric prioritization is insufficient. Prevention remains simple (prepared statements), but complementary controls carry real, workload-dependent costs, up to 49% throughput loss on small databases.

Results

Charts illustrate the learning outcomes before and after the implementation of digital tools.



Sources: OWASP Top 10 2013 (via 2017 Release Notes), 2017, 2021 and 2025 category pages, and the 2025 methodology introduction, owasp.org. Category scope changed between editions (33 CWEs in 2021, 37 in 2025), so CVE counts are indicative of scale, not strictly comparable.

Conclusion

SQL injection remains one of the highest volume vulnerability categories in modern software, with 62,445 CVEs recorded under OWASP's Injection classification, nearly twice as many as the next category. Detection is largely solved: a hybrid CNN-LSTM model trained on real network traffic reaches a 97% average F1-score. The real bottleneck is prioritization. Analyzing 317 SQL injection CVEs, Paul et al. found that CVSS alone flags 259 vulnerabilities for patching, many with limited exploitability, while EPSS alone catches only 19, leaving hundreds unaddressed. Their combined risk model narrows this to 153 vulnerabilities, cutting wasted remediation effort to near zero for high priority classes such as authentication bypass. Alongside this, a clear prevention hierarchy already exists, led by prepared statements with variable binding. Together, these results show that SQL injection risk can be measurably reduced without needing to fix every vulnerability at once.

Acknowledgement

We thank the original researchers of the study for their work. We are also grateful to Ms. Blumberger for her guidance, and to our peers for their feedback.

References

- https://pure.qub.ac.uk/files/612065296/SQL_injection_attack.pdf
- https://top10.owasp.org/2025/A05_2025-Injection/
- https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html